

LeWAM: END-TO-END JEPA FOR CONTROL WITH STOCHASTIC ACTION MODELING

Anonymous authors

Paper under double-blind review

[Website](#)

ABSTRACT

Joint-embedding predictive architectures (JEPAs) learn compact latent world models without pixel reconstruction. However, existing JEPAs typically decouple representation and dynamics learning from downstream control. Furthermore, their demonstrated scope remains confined to short-horizon tasks with limited behavioral complexity. We introduce **LeWAM**: an end-to-end JEPA with only two ingredients: (1) a joint prediction objective over latent states and actions that couples action-conditioned dynamics with stochastic action modeling, and (2) an isotropic Gaussian constraint on the latent states. Joint prediction supports direct control and efficient planning through latent imagination. With 17M parameters and single-GPU training, LeWAM delivers $10.8\times$ LeWM (CEM)’s planning throughput, with average success gains of 60.2 and 56.0 percentage points on contact-rich and 100-step tasks. Evaluation spans 8 simulated tasks and physical robots, with 30 Hz control versus GPT-6 Astra’s 1.4 Hz. Analyses assess control-centric representations and useful dynamics for task-agnostic goal reaching.

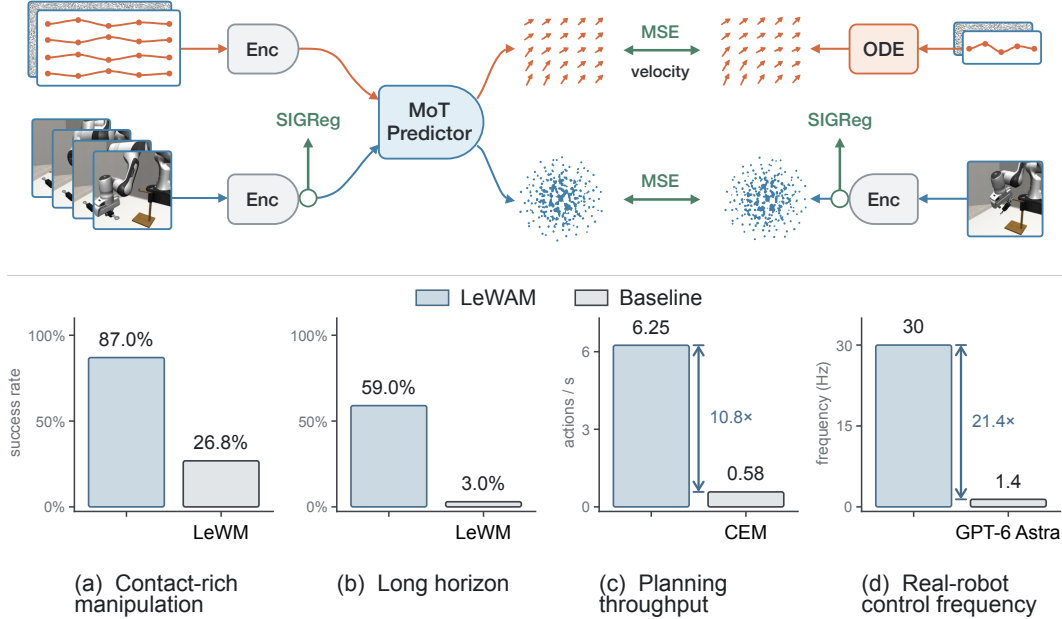


Figure 1: **LeWAM: joint prediction of latent states and actions.** **Top:** Four history frames and the future frame share an encoder; SIGReg (Balestriero & LeCun, 2025; Maes et al., 2026b) regularizes their latent states. The mixture of transformers (MoT) predictor (Liang et al., 2025) uses clean actions for MSE against the encoded future frame, and actions mixed with Gaussian noise to learn velocities through flow matching. Bold arrows distinguish losses from forward computation. **Bottom:** Goal-reaching success averaged over 4 diverse contact-rich manipulation tasks (a) and over 2 tasks at a 100-step goal horizon (b); planning throughput against LeWM (CEM) (c); and control frequency against GPT-6 Astra on physical robots (d).

LeWM, V-JEPA 2 ✗ Separated control	SIGReg, DINOv2 ✗ Generic representation	DreamZero, Fast-WAM ✗ Generative Video WAMs	CEM, MPPI, iCEM ✗ Population-based iterative search	Navigation, 2D Action ✗ Short-horizon simple tasks
LeWAM: End-to-End JEPA for Control with Stochastic Action Modeling				
✓ Unified world model and control	✓ Control-centric representation	✓ No reconstruction ✓ No pre-training	✓ Policy-guided search ✓ Gradient-based refinement	✓ Contact-rich ✓ Long horizon

Figure 2: **System-level positioning of LeWAM.** Compared with staged JEPA control pipelines, LeWAM unifies world modeling and control; compared with generic representation learning, it targets control-centric features; compared with generative video-based world action models, it uses a lightweight latent backbone without pixel reconstruction; and compared with search from broad priors, it combines learned action proposals with gradient-based search through latent imagination.

1 INTRODUCTION

Visual control requires autonomous agents to complete tasks by selecting actions based on visual observations of the world. World models (Ha & Schmidhuber, 2018) address this challenge by predicting the consequences of candidate actions, allowing agents to anticipate how the world will evolve before executing them. Originally developed for self-supervised visual representation learning (Assran et al., 2023; Bardes et al., 2024), joint-embedding predictive architectures (JEPAs) (LeCun, 2022) offer a compact world modeling paradigm where future world states are predicted in latent space rather than by reconstructing pixels. Recent works such as DINO-WM (Zhou et al., 2024) and LeWorldModel (LeWM) (Maes et al., 2026b) have demonstrated the promise of JEPAs for learning action-conditioned latent world models. However, using these models for control currently requires additional action solvers, such as CEM (Rubinstein, 1999), policy learning as in GC-IDM Nguyen et al. (2026), or both *e.g.* (Wang et al., 2026b). Furthermore, JEPA world models’ latent geometry is often shaped directly from the pretrained embeddings (Assran et al., 2025; Zhou et al., 2024) or by regularizers such as SIGReg (Balestriero & LeCun, 2025). While these priors produce informative embeddings, they do not explicitly shape the latent space around control-relevant directions. Thus, representation learning, world/dynamics modeling, and action selection remain separately optimized, creating a gap between the learned world model and the behavior required for control.

The resulting control mechanisms inherit these limitations. Population-based solvers such as CEM (Rubinstein, 1999), iCEM (Pinneri et al., 2021), and MPPI (Williams et al., 2017) enable planning with a learned dynamics model by sampling candidate action sequences and selecting the sequence the dynamics predicts will most likely achieve the desired goal. However, if candidate sequences are drawn from an uninformed prior, early samples can be concentrated far from useful behavior, initializing optimization in suboptimal regions of the action space, a problem which compounds with long-horizon contact-rich manipulations. Moreover, if the world model’s representations do not align with action-relevant directions, optimization can exploit spurious rather than useful features for downstream control.

We introduce **LeWAM**, a compact, end-to-end JEPA world action model that jointly learns visual representations, action-conditioned dynamics, and a stochastic policy from raw pixels in one training stage (Figure 1). Training uses two ingredients: a joint prediction objective over latent states and actions, and SIGReg (Balestriero & LeCun, 2025) to regularize the latent states. A shared mixture-of-transformers predictor (Liang et al., 2025) supports direct control and planning through latent imagination. Goal-reaching planning is our primary evaluation; behavior cloning additionally assesses the learned policy under direct execution. Our contributions are:

- An end-to-end JEPA for control that learns control-centric representations, action-conditioned dynamics, and a stochastic policy in one training stage.
- A fast planning algorithm for joint imaginations that initializes an action sequence from the informed action distribution and refines it through gradient descent.
- Across eight simulated environments and physical robot tasks, LeWAM delivers 10.8× the planning throughput of LeWM (CEM), with success-rate gains of 60.2 percentage points across four

contact-rich tasks and 56.0 points across two tasks at a 100-step goal horizon. Physical probes and dynamics diagnostics separately assess predictive scope.

2 RELATED WORK

Latent World Models. Latent world models support planning (Hafner et al., 2019) and policy learning in imagination (Hafner et al., 2020; 2023). Tokenized visual models (Micheli et al., 2023; Bruce et al., 2024) extend this approach to learned interactive environments. For task-directed control, TD-MPC (Hansen et al., 2022; 2024) shapes latent dynamics with reward, value, and policy objectives. LeWAM instead learns control-relevant representations from trajectories without reward or value supervision.

Joint-Embedding Predictive Architectures. JEPAs predict representations without reconstruction, using variance and covariance constraints (Bardes et al., 2022), momentum targets (Assran et al., 2023; Bardes et al., 2024), or distributional regularization (Balestriero & LeCun, 2025; Wu et al., 2026) to prevent collapse. For action-conditioned world modeling with JEPA, DINO-WM (Zhou et al., 2024) and V-JEPA 2-AC (Assran et al., 2025) build on pretrained representations, whereas LeWM (Maes et al., 2026b) leverages SIGReg (Balestriero & LeCun, 2025) to learn the encoder and latent dynamics jointly from scratch. Inverse-dynamics supervision also shapes representations, alongside distributional constraints in PLDM (Sobal et al., 2025) or as an anti-collapse signal in SMWM (Ivashkov et al., 2026). Action selection still requires a solver or post-trained policy. LeWAM retains distributional regularization while joint dynamics and stochastic action prediction supply control-relevant representation learning pressure.

Planning with Latent Dynamics. Online optimization and learned action inference provide two routes from prediction to control. CEM and MPPI (Rubinstein, 1999; Williams et al., 2017) search action populations initialized from some prior distribution *e.g.* Gaussian; iCEM (Pinneri et al., 2021) improves efficiency through correlated proposals and sample reuse. Value surrogates (Destrade et al., 2026) and hierarchical subgoals (Zhang et al., 2026; Masip et al., 2026) modify costs or shorten horizons while retaining population search. Gradient methods instead differentiate through the dynamics model to find optimal action sequences. Early gradient planners (Henaff et al., 2017) optimize actions without an action predictor, while Gradient MPC (Jyothir et al., 2023) uses Gaussian or separately learned policy initializations. CEM-GD (Huang et al., 2021) initializes gradient descent with action sequences found by CEM. Parthasarathy et al. (2025) address model error under optimized actions through simulator-corrected rollouts or adversarial state and action perturbations; latent trajectory straightening (Wang et al., 2026a) improves optimization geometry. GPC (Brudermüller et al., 2025) uses flow-matching proposals to guide CEM in contact-rich MPC. Post-trained action models (Nguyen et al., 2026; Wang et al., 2026b) instead amortize proposals: GC-IDM acts directly, whereas PRISM retains MPPI refinement. By jointly learning the policy and dynamics, LeWAM natively samples and evaluates policy proposals and further refines these rollouts with gradient updates instead of repeated population search.

World Action Models. WAMs couple world prediction with action generation (Table 5, Appendix B.2). Built on pretrained video models, DreamZero (Ye et al., 2026), Fast-WAM (Yuan et al., 2026) jointly predict the next action chunk and video frame, with the latter skipping video generation during action inference. UVA (Li et al., 2025) and UWM (Zhu et al., 2025) specifically support forward dynamics by making world prediction action conditioned. Cosmos 3 (NVIDIA, 2026) predicts a diverse observation space including language, audio, camera motion, and egocentric pose. LeWAM jointly trains its encoder and predictor from scratch without pixel reconstruction. Each training sample supplies clean actions for latent dynamics and noisy actions for stochastic action prediction. This split allows dynamics learning from both demonstrations and non-expert interactions.

3 METHODOLOGY

LeWAM learns the joint prediction of latent states and actions from reward-free offline trajectories. A shared predictor combines action-conditioned dynamics with a stochastic policy, supporting both direct control and planning through imagined consequences.

3.1 LEARNING THE JOINT PREDICTOR

Setting. Training windows $\mathcal{D}$ come from trajectories with recorded actions, including non-expert interactions. Let $\mathbf{z}_t = \text{enc}(\mathbf{o}_t)$ encode observation $\mathbf{o}_t$ and $\mathbf{z}_{\leq t}$ denote the fixed encoded history. The default simulation model uses 192-dimensional latent states. We omit network-parameter subscripts throughout. All time indices refer to raw environment steps: $\mathbf{a}_t$ transitions $\mathbf{o}_t$ to $\mathbf{o}_{t+1}$. Write $\mathbf{A}_t = (\mathbf{a}_t, \dots, \mathbf{a}_{t+K-1})$ for the K -action chunk beginning at t . Dynamics predicts $\mathbf{z}_{t+K}$ from $\mathbf{A}_t$, while the policy generates an action chunk $\mathbf{a}$ of size $\geq K$. Let $\mathbf{q}_{t+K}^z$ denote the learned state-query token for predicting $\mathbf{z}_{t+K}$. Frame skipping applies only to observations; actions are neither subsampled nor averaged. For goal-reaching training, a future observation from the same trajectory is randomly sampled as the goal, without task-completion labels. Optional context $\mathbf{c}_t$ contains the encoded goal $\mathbf{z}_g$ and normalized remaining horizon; $\mathbf{c}_t$ is omitted for goal-free control.

LeWAM Architecture. Figure 1 summarizes the model. In the default simulation model, enc uses a ResNet-18-style backbone (He et al., 2016), spatial softmax pooling, and a two-layer projection MLP to produce the latent state. The same encoder processes history, future target, and goal images. A mixture-of-transformers (MoT, Liang et al. (2025)) predictor cleanly balances stochastic action modeling and action-conditioned forward dynamics. The encoder and predictor are trained jointly from scratch, with encoded future images providing latent regression targets.

The state branch f_z and action branch f_a of the MoT predictor output future latent states and raw-action velocities, respectively. Write $\mathbf{a}^{(s)}$ for the noisy action chunk at flow time $s \in [0, 1]$:

$$\begin{aligned}\hat{\mathbf{z}}_{t+K} &= f_z(\mathbf{z}_{\leq t}, \mathbf{A}_t), \\ \hat{\mathbf{v}} &= f_a(\mathbf{z}_{\leq t}, \mathbf{a}^{(s)}, s; \mathbf{c}_t).\end{aligned}\tag{1}$$

Integrating these velocities samples an action chunk (Section 3.2).

Linear input projections map latent states and actions to a common token width. The sequence contains history, $\mathbf{q}_{t+K}^z$, K clean action tokens, and $2K$ noisy action-flow tokens, distinguished by learned type and temporal embeddings. Each MoT layer applies separate state and action attention projections and feedforward transformations while sharing one masked self-attention operation. Flow timesteps modulate only action-flow tokens through AdaLN-Zero (Peebles & Xie, 2023).

Figure 3 specifies the information available to each prediction. The state query $\mathbf{q}_{t+K}^z$ reads history and clean actions; action-flow tokens read history and their causal prefix. History attends only to history, blocking indirect clean-action leakage. Goal and horizon condition only the action readout, leaving the MoT attention and dynamics unconditioned on either. Appendix B gives the full token-level mask, layer sizes, and training settings.

Training Objective. For flow matching (Lipman et al., 2023), given a recorded action chunk $\mathbf{a}$, sample s uniformly from $[0, 1]$ and $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, then construct

$$\mathbf{a}^{(s)} = (1 - s)\epsilon + s\mathbf{a}, \quad \frac{d\mathbf{a}^{(s)}}{ds} = \mathbf{a} - \epsilon.\tag{2}$$

The action readout regresses the path velocity $\mathbf{a} - \epsilon$ from $\mathbf{a}^{(s)}$. The dynamics and action losses use MSE, averaged over the scalar coordinates of each prediction:

$$\begin{aligned}\mathcal{L}_{\text{dyn}} &= \mathbb{E}_{\mathcal{D}}[\text{MSE}(\hat{\mathbf{z}}_{t+K}, \mathbf{z}_{t+K})], \\ \mathcal{L}_{\text{act}} &= \mathbb{E}_{\mathcal{D}, s, \epsilon}[\text{MSE}(\hat{\mathbf{v}}, \mathbf{a} - \epsilon)].\end{aligned}\tag{3}$$

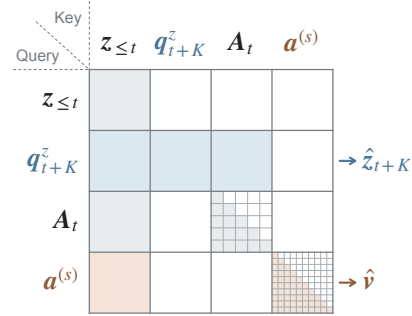


Figure 3: **MoT attention.** Filled cells permit attention; white cells are masked, and fine grids show causal attention to the right of prediction-query rows show their readouts.

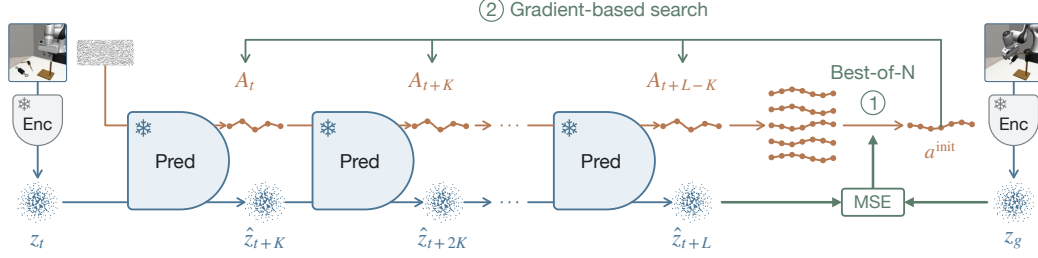


Figure 4: **Planning with LeWAM.** Each MoT predictor stage samples $2K$ actions from Gaussian noise and uses the first K , denoted A_{t+k} , to advance the latent dynamics from $t+k$ by K raw environment steps. Five-point curves depict action chunks ($K = 5$); stacked curves denote N candidate plans of length L . Global search in the action distribution (step 1) performs only one iteration of population-based search, selecting the lowest-MSE plan as a^{init} . Gradient-based search (step 2) updates a^{init} by differentiating through the frozen dynamics with respect to the constrained objective in Appendix D.2.

We use SIGReg (Balestriero & LeCun, 2025; Maes et al., 2026b) to preserve latent diversity and discourage overspecialization to training action trajectories. Let Z collect the encoded history and target observations in a minibatch. With M random unit directions $u^{(m)}$ and the univariate Epps–Pulley (Epps & Pulley, 1983) test statistic T against $\mathcal{N}(0, 1)$, the regularizer is

$$\mathcal{L}_{sig} = \frac{1}{M} \sum_{m=1}^M T(Zu^{(m)}). \quad (4)$$

With regularization weight λ_{sig} , the full objective is

$$\mathcal{L}_{LeWAM} = \mathcal{L}_{dyn} + \mathcal{L}_{act} + \lambda_{sig} \mathcal{L}_{sig}. \quad (5)$$

3.2 CONTROL AND PLANNING WITH LEWAM

LeWAM supports direct control through its learned policy and planning through imagined futures.

Control. With history and optional goal context fixed, one $2K$ -action chunk is sampled by integrating the learned velocity field from Gaussian noise:

$$\frac{da^{(s)}}{ds} = f_a(z_{\leq t}, a^{(s)}, s; c_t), \quad a^{(0)} \sim \mathcal{N}(0, I), \quad \hat{a} = a^{(1)}. \quad (6)$$

The first K actions are executed and resampling begins from the resulting observation.

Planning. LeWAM uses a global-to-local strategy (Figure 4): *only one iteration of population-based search* per planning call, followed by gradient-based search. We let a denote a plan to reach the goal z_g after L raw environment steps from time t , where L is a multiple of K .

① **Global Search in the Action Distribution.** Starting from the observed history $\hat{z}_{\leq t} = z_{\leq t}$, one population of N candidate plans is generated from the learned action distribution by alternating action sampling and latent dynamics. Specifically, at each step, Eq. (6) generates N $2K$ -action chunks from the observation history and goal. The forward dynamics branch uses the first K actions of each chunk to predict the N corresponding next states. Each predicted state is appended to the fixed-length history of its candidate, the planning horizon is updated, and the process continues for L total steps. A single best-of- N selection chooses the plan with the lowest latent goal cost:

$$\mathcal{C}(a) = \text{MSE}(\hat{z}_{t+L}(a), z_g), \quad a^{init} = \arg \min_{a \in \{a^{(n)}\}_{n=1}^N} \mathcal{C}(a). \quad (7)$$

② **Gradient-Based Search.** Temporally coherent policy rollouts provide a structured initialization; we assume that nearby improvements can be found through gradients of the latent dynamics.

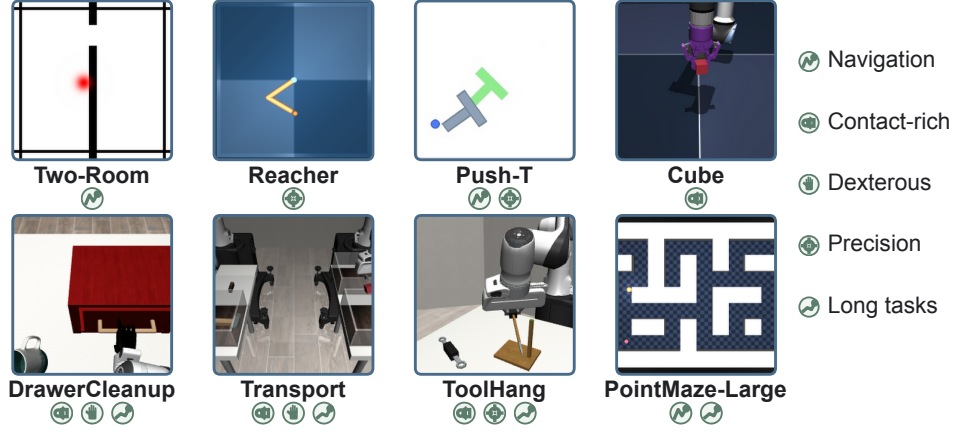


Figure 5: **Representative simulation environments.** The first row shows four environments used by LeWM; the second row introduces complementary navigation and manipulation settings. Symbols beneath each frame denote overlapping task attributes. The *Long tasks* label refers to full task duration; goal reaching uses a fixed horizon.

Starting from $\mathbf{a}^{\text{init}}$, actions can be refined through the frozen dynamics model with gradient-based planning (Henaff et al., 2017; Jyothir et al., 2023). With learning rate η , the goal-cost update takes the gradient-descent form

$$\mathbf{a}_{t+k} \leftarrow \mathbf{a}_{t+k} - \eta \frac{\partial \mathcal{C}}{\partial \mathbf{a}_{t+k}}, \quad k = 0, \dots, L-1, \quad (8)$$

The descent iterate with the lowest $\mathcal{C}$, including $\mathbf{a}^{\text{init}}$, is chosen. Note that in practice, we add a small trust-region penalty to $\mathcal{C}$ to avoid model-exploitation. Appendix D.2 details the Adam implementation, proximity penalty, and clipping.

Optionally, the first K actions of a plan are executed, and the entire two-stage process repeats until the goal is reached as in classical receding-horizon MPC.

4 EXPERIMENTS

Our primary evaluation studies goal-reaching planning. We additionally evaluate behavior cloning to assess the quality of the learned policy under direct execution. Physical probes assess control-centric representations, dynamics diagnostics test action consequences, and planning analyses evaluate their use in goal-directed action selection.

4.1 TOWARDS GENERAL-PURPOSE JEPA CONTROL IN SIMULATION

We evaluate general-purpose JEPA control across eight simulated navigation and manipulation environments (Figure 5). The environments vary in task functionality, temporal horizon, data quality and collection regime, and the number of camera views, covering expert demonstrations, random-policy interactions, and mixed-quality data. Four environments adapt the LeWM (Maes et al., 2026b) benchmarks; four add complementary manipulation and navigation challenges. Appendix C summarizes task choices and observations in Table 6, with dataset properties in Table 7. Appendix D gives the evaluation protocol and control checks.

Evaluation Protocols. We distinguish two settings. Goal reaching (GR) assumes an available goal image: a goal-conditioned policy or planner must reach the depicted state within a fixed horizon. Task completion (TC) provides no goal image and evaluates behavior cloning from demonstrations: policies execute actions from observation history to complete the task. Here LeWAM uses direct control with $\mathbf{c}_t$ omitted. Within each setting, methods share observations, action limits, execution budgets, and success predicates. Appendix D.1 details goal construction and success criteria.

Compared Methods. For goal-conditioned evaluation, we compare the following configurations. FDM denotes forward dynamics conditioned on supplied actions; Policy denotes learned action prediction. End-to-end indicates joint optimization of the visual encoder and task prediction modules. Method colors match the result figures; Appendix B.1 gives implementation details.

Method	Introduction	FDM	Policy	End-to-end
● LeWAM (ours)	Joint prediction of latent states and actions for direct control and gradient-based planning.	✓	✓	✓
● LeWM (IDM) (Nguyen et al., 2026)	Post-trains a goal-conditioned IDM on frozen LeWM.	✓	✓	✗
● LeWM (CEM) (Maes et al., 2026b)	Learns latent dynamics with SIGReg; selects actions with CEM.	✓	✗	✓
● DINO-WM (Zhou et al., 2024)	Learns dynamics over frozen DINOv2 patch features; plans with CEM.	✓	✗	✗
● GCBC (Ghosh et al., 2021)	Goal-conditioned policy trained on frozen DINOv2 features.	✗	✓	✗

4.2 RESULTS ON GOAL REACHING

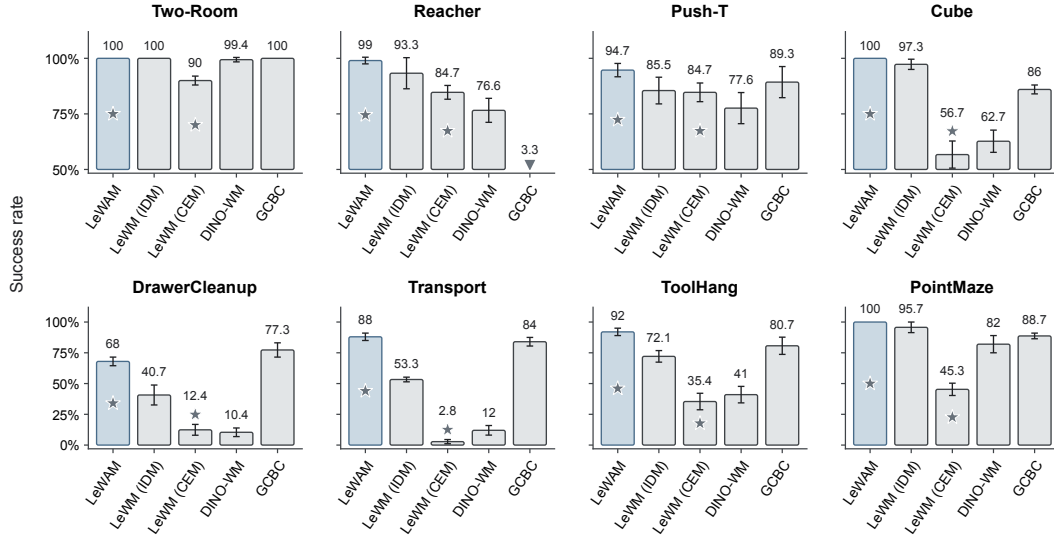


Figure 6: **Goal reaching.** Each panel denotes one environment under the common short-horizon protocol detailed in Appendix D. Filled bars report current measurements. Error bars denote one standard deviation; ★ indicates end-to-end training. Linear axes start at 50% in the top row and 0% in the bottom row. The downward marker retains Reacher GCBC ($3.3 \pm 2.3\%$), below the displayed range. Only the LeWAM results on DrawerCleanup and Transport use earlier measurements pending final results.

On Push-T, at a 100-step goal horizon, LeWAM achieves 45.3% success compared with 2.7% for LeWM (CEM) (Maes et al., 2026b), a gap of 42.6 percentage points (Figure 7).

Appendix E.1 reports start-to-end goal reaching, where the trajectory’s final image supplies the goal.

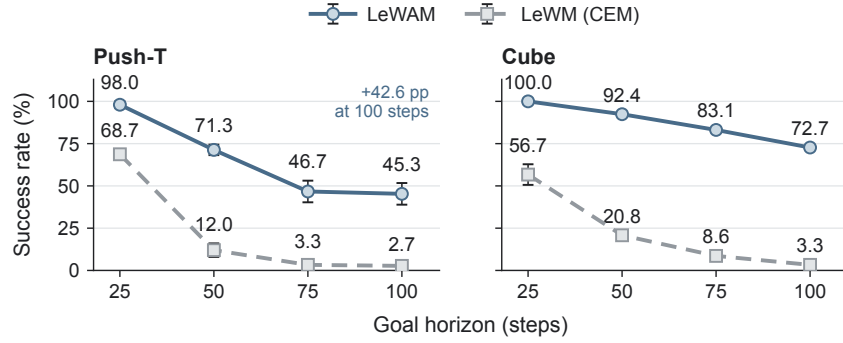


Figure 7: **Goal-reaching success across horizons.** *Push-T* starts at frame 0, uses twice the goal horizon in actions, and replans every 25 steps. Error bars show one standard deviation. *Cube* combines 25-step records, a preliminary zero-shot LeWAM estimate at 100 steps, and visual placeholders elsewhere. Appendix D.3 gives protocols and values.

4.3 RESULTS ON TASK COMPLETION

For goal-free task completion, we mask the goal context c_t during joint training, so the action branch predicts $\hat{v} = f_a(z_{\leq t}, a^{(s)}, s)$ while retaining the JEPA dynamics objective. At evaluation, the policy samples actions from observation history without c_t and executes them directly, without planning. The comparison with Diffusion Policy (Chi et al., 2023) in Table 1 tests whether jointly learning goal-agnostic dynamics benefits stochastic policy learning.

Table 1: **Goal-free task completion.** Mean success rates (%) $\pm$ standard deviation, using observation history without a goal image or goal-horizon input (Appendix E.2). LeWAM uses one camera. Expert replay reaches 72.0% on *DrawerCleanup*; dashes denote pending results.

Method	Cube	DrawerCleanup	Transport	ToolHang
Diffusion Policy (Chi et al., 2023)	–	54.7 $\pm$??	45.3 $\pm$??	84.7 $\pm$ 3.1
LeWAM	–	52.7 $\pm$??	61.3 $\pm$??	80.0 $\pm$ 3.5

4.4 UNDERSTANDING THE LEARNED REPRESENTATION

Can joint prediction prevent representation collapse? We first test whether joint prediction of latent states and actions can maintain an informative representation without explicit distribution regularization. The decisive comparison is between parameter-matched latent-only and joint models with SIGReg disabled in both. Matched physical probes, planning outcomes, and learning curves remain pending. *Placeholder*

Physical-state linear probing. We measure how well linear probes recover physical states from LeWAM’s representations, while ablating SIGReg (Balestriero & LeCun, 2025; Maes et al., 2026b) to assess its effect on the representation. With SIGReg, probe R^2 reaches 0.947 on *Push-T* and 0.929 on *ToolHang*, compared with 0.803 and 0.868 without it (Table 2). Effective rank also increases. Planning success increases by 16.0 and 2.6 percentage points, respectively; direct success on *ToolHang* remains 92.0%. Table 3(a) isolates the gain from adding planning to direct control of the same model. Action-conditioned dynamics and held-out-goal diagnostics remain pending.

Table 2: Physical-state linear probing and SIGReg ablation. LeWAM probes decode the Push-T block and ToolHang tool states and report R^2 . LeWM’s published Push-T value reports block-angle Pearson r (Maes et al., 2026b); its planning SRs are from Figure 6. Effective rank uses the participation ratio. Dashes denote unavailable entries.

Environment	Model	SIGReg	Eff. rank	Linear probe	Direct SR (%)	Plan SR (%)
Push-T	LeWAM	Yes	32.71	0.947	80.0	94.7
	LeWAM	No	4.72	0.803	72.7	78.7
	LeWM	Yes	–	0.902 (r)	–	84.7
ToolHang	LeWAM	Yes	19.75	0.929	92.0	93.3
	LeWAM	No	8.20	0.868	92.0	90.7
	LeWM	Yes	–	–	–	35.4

Does LeWAM learn a control-centric representation? Here control-centric means that visual features adapt to the distinctions needed to predict actions and their consequences. We compare the learned encoder with frozen DINOv2 CLS and patch features while retaining the joint predictor. Table 3(b) uses the same tasks and goal-reaching protocol as the planning ablation; matched control results and physical probes remain pending. The learned latent-only control further tests the contribution of action supervision. Appearance and geometry shifts test whether features respond selectively to changes in action consequences. *Placeholder*.

Table 3: Planning and visual encoder ablations. Both blocks use the same short-horizon goal-reaching tasks. (a) Direct and planning SR (%) for the same LeWAM checkpoint (Table 2); Gain is Planning minus Direct in percentage points. (b) Planning SR (%) with learned or frozen visual encoders and the same joint predictor. Dashes denote pending matched results.

Configuration	Push-T	Cube	ToolHang
(a) Effect of planning			
Direct SR (%)	80.0	–	92.0
Planning SR (%)	94.7	–	93.3
Gain (pp)	+14.7	–	+1.3
(b) Visual encoder ablations			
LeWAM (learned encoder)	–	–	–
frozen DINOv2 CLS	–	–	–
frozen DINOv2 patch	–	–	–

4.5 ACTION TARGETS AND SEARCH

Figure 8 illustrates action-head behavior in a two-door navigation task. The demonstrations cover both passages. The MSE rollouts shown concentrate on the lower passage and include collisions with the central wall, whereas the flow-generated rollouts traverse both passages.

The FM versus action-MSE ablation asks whether a learned action distribution supplies distinct useful candidates for population-based search, and how their initializations affect subsequent gradient-based search. It replaces only the action target, retaining state MSE and SIGReg. Matched action-target comparisons remain pending. *Placeholder*.

Planning throughput. Figure 1(c) reports 6.25 actions/s for LeWAM and 0.58 actions/s for LeWM (CEM) (Maes et al., 2026b), computed as the reciprocals of 160 and 1,727 ms per action, respectively. This corresponds to a $10.8\times$ speedup under the reported configurations. Appendix D.3 records the timing scope.

4.6 RESULTS ON PHYSICAL ROBOTS

The original DINO-WM (Zhou et al., 2024), PLDM (Sobal et al., 2025), and LeWM (Maes et al., 2026b) studies report control in simulation rather than closed-loop physical-robot deployment. Our

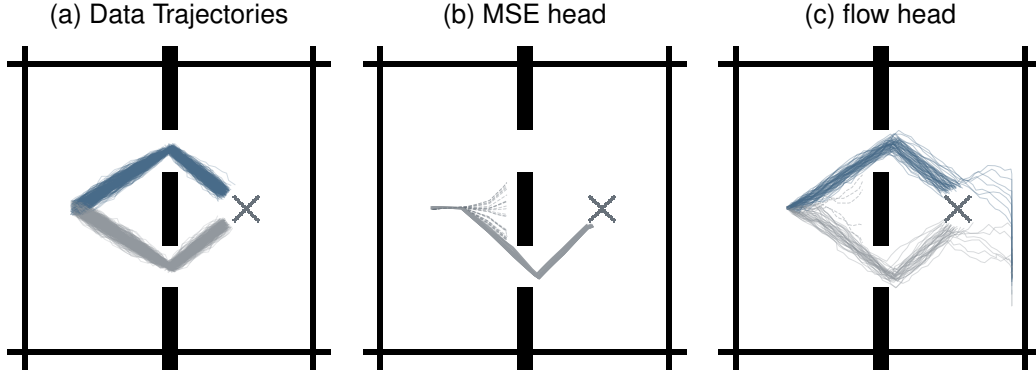


Figure 8: **Action-head rollouts in a two-door environment.** Planar agent trajectories from the demonstration dataset (a), the deterministic MSE action head (b), and the flow-matching action head (c). Black segments are walls and the dark gray cross is the goal. Blue and gray indicate upper- and lower-door assignments; dashed dark gray paths denote rollouts ending in collision. **TODO: tool hang or any manipulation**

robot study asks whether this reconstruction-free predictive paradigm can transfer to physical manipulation.

We evaluate both an end-to-end model and a variant built on a frozen V-JEPA 2.1 backbone (Mur-Labadia et al., 2026) on Pick and Place T and Push-T, with a third task marked TBD. We compare against Diffusion Policy (Chi et al., 2023), $\pi_{0.5}$ (Black et al., 2025), and GPT-6 Astra.

Appendix F gives the hardware, data collection, model interfaces, and evaluation procedure.

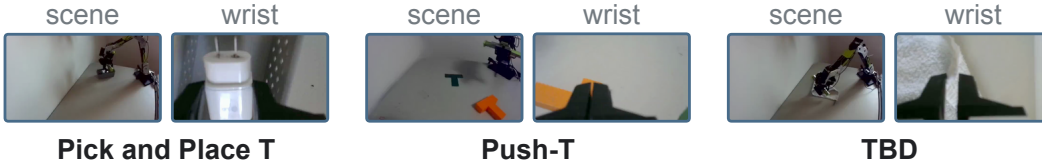


Figure 9: Physical-robot task suite. Paired scene and wrist views use temporary example frames pending task-specific replacements. The right stereo view is recorded but is not used by the policy.

Table 4: **Physical robot evaluation.** Successful trials out of 30 on Pick and Place T and Push-T; the third task remains TBD. GPT-6 Astra uses one demonstration video, with a goal image specified for each evaluation episode. Cost reports training or post-training time (h), or API spend (US\$) for GPT-6 Astra. Missing entries await finalized evaluation.

Method	Pick and Place T	Push-T	TBD	Cost
Diffusion Policy (Chi et al., 2023)	19/30	16/30	TBD	TBD h
$\pi_{0.5}$ (Black et al., 2025)	14/30	TBD	TBD	TBD h
GPT-6 Astra (one-shot + goal image)	TBD	TBD	TBD	US\$ TBD
LeWAM (V-JEPA 2.1)	15/30	12/30	TBD	TBD h
LeWAM (end to end)	16/30	8/30	TBD	TBD h

5 CONCLUSION AND LIMITATIONS

We introduced LeWAM, an end-to-end JEPA for joint prediction of latent states and actions, with goal-reaching planning as the primary evaluation. Behavior cloning provides a supplementary as-

assessment of the learned policy under direct execution. Training spans demonstrations and non-expert interactions, with separate models evaluated for each task and embodiment in simulation and on physical robots. Appendix A documents reproducibility and asset licensing.

Limitations and Future Work.

- **Data coverage.** Complex contact-rich manipulation still relies on demonstrations. Whether broader suboptimal interaction data improves dynamics and physical understanding remains unverified and is a direction for future work.
- **Representation scope.** Physical decodability does not ensure reliable control under distribution shift. Distinctions absent from both recorded actions and their consequences may be compressed; joint supervision does not exclude partial collapse or contextual shortcuts.
- **Single-task and single-embodiment learning.** Goal-independent dynamics does not establish transfer of one model across tasks or embodiments. SIGReg (Balestriero & LeCun, 2025; Maes et al., 2026b) constrains each task’s latent distribution without aligning coordinates across domains; such alignment remains untested.
- **Integration with hierarchical planning.** LeWAM provides low-level control and does not include a global high-level planner for very long-horizon tasks. Future work could integrate it with an external planner that decomposes such tasks into subgoal sequences, with LeWAM executing each subgoal through continuous control.

REFERENCES

- Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, et al. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025.
- Randall Balestriero and Yann LeCun. LeJEPA: Provable and scalable self-supervised learning without the heuristics. *arXiv preprint arXiv:2511.08544*, 2025.
- Adrien Bardes, Jean Ponce, and Yann LeCun. VICReg: Variance-invariance-covariance regularization for self-supervised learning. In *International Conference on Learning Representations (ICLR)*, 2022.
- Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video. *arXiv preprint arXiv:2404.08471*, 2024.
- Kevin Black et al. $\pi_{0.5}$: A vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- Jake Bruce, Michael Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, et al. Genie: Generative interactive environments. In *International Conference on Machine Learning (ICML)*, 2024.
- Lara Brudermüller, Brandon Hung, Xinghao Zhu, Jiuguang Wang, Nick Hawes, Preston Culbertson, and Simon Le Cleac’h. Generative models from and for sampling-based MPC: A bootstrapped approach for adaptive contact-rich manipulation. *arXiv preprint arXiv:2510.14643*, 2025. URL <https://arxiv.org/abs/2510.14643>.
- Remi Cadene, Simon Aliberts, Francesco Capuano, Michel Aractingi, Adil Zouitine, Pepijn Kooijmans, Quentin Gallouédec, Thomas Wolf, et al. LeRobot: An open-source library for end-to-end robot learning. *arXiv preprint arXiv:2602.22818*, 2026.
- Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems (RSS)*, 2023.

-
- Matthieu Destrade, Oumayma Bounou, Quentin Le Lidec, Jean Ponce, and Yann LeCun. Value-guided action planning with JEPA world models. *arXiv preprint arXiv:2601.00844*, 2026.
- Thomas W Epps and Lawrence B Pulley. A test for normality based on the empirical characteristic function. *Biometrika*, 70(3):723–726, 1983.
- Dibya Ghosh, Abhishek Gupta, Ashwin Reddy, Justin Fu, Coline Devin, Benjamin Eysenbach, and Sergey Levine. Learning to reach goals via iterated supervised learning. In *International Conference on Learning Representations (ICLR)*, 2021.
- David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018.
- Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *International Conference on Machine Learning (ICML)*, 2019.
- Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations (ICLR)*, 2020.
- Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.
- Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. In *International Conference on Learning Representations (ICLR)*, 2024.
- Nicklas A. Hansen, Hao Su, and Xiaolong Wang. Temporal difference learning for model predictive control. In *International Conference on Machine Learning (ICML)*, pp. 8387–8406, 2022.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016. URL <https://arxiv.org/abs/1512.03385>.
- Mikael Henaff, William F. Whitney, and Yann LeCun. Model-based planning with discrete and continuous actions. *arXiv preprint arXiv:1705.07177*, 2017.
- Kevin Huang, Sahin Lale, Ugo Rosolia, Yuanyuan Shi, and Anima Anandkumar. CEM-GD: Cross-entropy method with gradient descent planner for model-based reinforcement learning. *arXiv preprint arXiv:2112.07746*, 2021. URL <https://arxiv.org/abs/2112.07746>.
- Petr Ivashkov, Randall Balestriero, and Bernhard Schölkopf. Sensorimotor world models: Perception for action via inverse dynamics. *arXiv preprint arXiv:2606.20104*, 2026.
- Zhenyu Jiang, Yuqi Xie, Kevin Lin, Zhenjia Xu, Weikang Wan, Ajay Mandlekar, Linxi Fan, and Yuke Zhu. DexMimicGen: Automated data generation for bimanual dexterous manipulation via imitation learning. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2025.
- S V Jyothir, Siddhartha Jalagam, Yann LeCun, and Vlad Sobal. Gradient-based planning with world models. *arXiv preprint arXiv:2312.17227*, 2023.
- Yann LeCun. A path towards autonomous machine intelligence. *OpenReview [Position Paper]*, 2022. URL <https://openreview.net/pdf?id=BZ5alr-kVsf>. Version 0.9.2.
- Shuang Li, Yihuai Gao, Dorsa Sadigh, and Shuran Song. Unified video action model. *arXiv preprint arXiv:2503.00200*, 2025.
- Weixin Liang, Lili Yu, Liang Luo, Srinivasan Iyer, Ning Dong, Chunting Zhou, Gargi Ghosh, Mike Lewis, Wen tau Yih, Luke Zettlemoyer, and Xi Victoria Lin. Mixture-of-transformers: A sparse and scalable architecture for multi-modal foundation models, 2025. URL <https://arxiv.org/abs/2411.04996>.
- Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023.

-
- Lucas Maes, Quentin Le Lidec, Luiz Facury, Nassim Massaudi, Ayush Chaurasia, Francesco Capuano, Richard Gao, Taj Gillin, Dan Haramati, Damien Scieur, Yann LeCun, and Randall Balestriero. stable-worldmodel: A platform for reproducible world modeling research and evaluation. *arXiv preprint arXiv:2605.21800*, 2026a.
- Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. LeWorld-Model: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026b.
- Ajay Mandlekar, Danfei Xu, Josiah Wong, Soroush Nasiriany, Chen Wang, Rohun Kulkarni, Li Fei-Fei, Silvio Savarese, Yuke Zhu, and Roberto Martín-Martín. What matters in learning from offline human demonstrations for robot manipulation. In *Conference on Robot Learning (CoRL)*, 2021.
- Sergi Masip, Jonathan Swinnen, Yutong Hu, Renaud Detry, and Tinne Tuytelaars. FF-JEPA: Long-horizon planning in world models with latent planners. *arXiv preprint arXiv:2606.09311*, 2026.
- Vincent Micheli, Eloi Alonso, and François Fleuret. Transformers are sample-efficient world models. In *International Conference on Learning Representations (ICLR)*, 2023.
- Lorenzo Mur-Labadia, Matthew Muckley, Amir Bar, Mahmoud Assran, Koustuv Sinha, Michael Rabbat, Yann LeCun, Nicolas Ballas, and Adrien Bardes. V-JEPA 2.1: Unlocking dense features in video self-supervised learning. *arXiv preprint arXiv:2603.14482*, 2026.
- Hoang Nguyen, Xiaohao Xu, and Xiaonan Huang. Latent geometry beyond search: Amortizing planning in world models. *arXiv preprint arXiv:2605.08732*, 2026.
- NVIDIA. Cosmos 3: Omnimodal world models for physical AI. *arXiv preprint arXiv:2606.02800*, 2026.
- Seohong Park, Kevin Frans, Benjamin Eysenbach, and Sergey Levine. OGBench: Benchmarking offline goal-conditioned RL. In *International Conference on Learning Representations (ICLR)*, 2025.
- Arjun Parthasarathy, Nimit Kalra, Rohun Agrawal, Yann LeCun, Oumayma Bounou, Pavel Izmailov, and Micah Goldblum. Closing the train-test gap in world models for gradient-based planning. *arXiv preprint arXiv:2512.09929*, 2025.
- William Peebles and Saining Xie. Scalable diffusion models with transformers. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- Cristina Pinneri, Shambhuraj Sawant, Sebastian Blaes, Jan Achterhold, Joerg Stueckler, Michal Rolinek, and Georg Martius. Sample-efficient cross-entropy method for real-time planning. In *Proceedings of the 2020 Conference on Robot Learning*, volume 155 of *Proceedings of Machine Learning Research*, pp. 1049–1065. PMLR, 2021. URL <https://proceedings.mlr.press/v155/pinneri21a.html>.
- Reuven Rubinstein. The cross-entropy method for combinatorial and continuous optimization. *Methodology and Computing in Applied Probability*, 1:127–190, 1999.
- Vlad Sobal, Wancong Zhang, Kyunghyun Cho, Randall Balestriero, Tim G. J. Rudner, and Yann LeCun. Learning from reward-free offline data: A case for planning with latent dynamics models. *arXiv preprint arXiv:2502.14819*, 2025.
- Ying Wang, Oumayma Bounou, Gaoyue Zhou, Randall Balestriero, Tim G. J. Rudner, Yann LeCun, and Mengye Ren. Temporal straightening for latent planning. *arXiv preprint arXiv:2603.12231*, 2026a.
- Yuhai Wang, Jiawei Xia, Rongxuan Zhou, Xiao Hu, Yongliang Shi, Jing Du, and Yang Ye. PRISM: PRior-guided Imagination Sampling in world Models. *arXiv preprint arXiv:2606.07974*, 2026b.
- Grady Williams, Nolan Wagener, Brian Goldfain, Paul Drews, James M. Rehg, Byron Boots, and Evangelos A. Theodorou. Information theoretic MPC for model-based reinforcement learning. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2017.

Haiyu Wu, Randall Balestriero, and Morgan Levine. VISReg: Variance-invariance-sketching regularization for JEPA training. *arXiv preprint arXiv:2606.02572*, 2026.

Seonghyeon Ye, Yunhao Ge, Kaiyuan Zheng, Shenyuan Gao, Sihyun Yu, et al. World action models are zero-shot policies. *arXiv preprint arXiv:2602.15922*, 2026.

Tianyuan Yuan, Zibin Dong, Yicheng Liu, and Hang Zhao. Fast-WAM: Do world action models need test-time future imagination? *arXiv preprint arXiv:2603.16666*, 2026.

Wancong Zhang, Basile Terver, Artem Zhohus, Soham Chitnis, Harsh Sutaria, Mido Assran, et al. Hierarchical planning with latent world models. *arXiv preprint arXiv:2604.03208*, 2026.

Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. DINO-WM: World models on pre-trained visual features enable zero-shot planning. *arXiv preprint arXiv:2411.04983*, 2024.

Chuning Zhu, Raymond Yu, Siyuan Feng, Benjamin Burchfiel, Paarth Shah, and Abhishek Gupta. Unified world models: Coupling video and action diffusion for pretraining on large robotic datasets. *arXiv preprint arXiv:2504.02792*, 2025.

756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809

Appendix

Supplementary material for *LeWAM: End-to-End JEPA for Control with Stochastic Action Modeling*

Roadmap

A	Reproducibility, Ethics, and Safety	p. 16
B	Implementation Details	p. 16
	Baseline Implementations	p. 17
	System-Level Simplicity and Cost	p. 18
C	Datasets and Benchmarks	p. 19
D	Evaluation Protocol	p. 20
	Goals and Success Criteria	p. 20
	LeWAM Planning	p. 21
	Goal-Horizon Evaluation and Timing	p. 22
E	Additional Results	p. 23
	Start-to-End Goal Reaching	p. 23
	Behavior Cloning	p. 23
F	Physical-Robot Setup and Evaluation	p. 24
	Platform and Data Collection	p. 24
	Model Interfaces	p. 24
	Closed-Loop Evaluation	p. 24

A REPRODUCIBILITY, ETHICS, AND SAFETY

Reproducibility. The experiments in Section 4 use the architectures and baseline configurations in Appendix B, datasets in Appendix C, and evaluation protocols in Appendix D. The artifact will release code, configurations, data-processing scripts, result records, and redistributable checkpoints.

Ethics and licensing. The experiments collect no new human-subject data. Reused environments, datasets, models, and software remain subject to their original licenses; restricted assets will be accompanied by acquisition instructions. Physical deployment (Section 4.6) requires independent safety mechanisms.

B IMPLEMENTATION DETAILS

Temporal Sampling. With observation-history stride Δ , the predictor in Section 3.1 uses two history latents $\mathbf{z}_{\leq t} = (\mathbf{z}_{t-\Delta}, \mathbf{z}_t)$, a dynamics interval of $K = 5$ raw steps, and an action chunk of $2K = 10$ raw steps. The stride is $\Delta = K$ for strided goal-reaching data and $\Delta = 1$ for consecutive task-completion frames. To predict $\mathbf{z}_{t+5}$, dynamics receives all five actions $\mathbf{a}_t, \dots, \mathbf{a}_{t+4}$; the action flow models all ten actions $\mathbf{a}_t, \dots, \mathbf{a}_{t+9}$. Only frames are subsampled. During imagined rollouts, the history update retains the two most recent latent states.

Inner Architecture. Unless a task-specific deviation is stated, the observation encoder is a ResNet-18-style network (He et al., 2016) trained from scratch. Its convolutional backbone uses GroupNorm with 16 groups. Input images are cropped from 224×224 to 202×202 , with random offsets during training and a center crop at evaluation. A 1×1 convolution produces 32 spatial maps; spatial softmax pooling converts these to 32 expected normalized image-coordinate pairs. A two-layer projection MLP with BatchNorm maps these 64 features through a hidden width of 384 to the 192-dimensional latent state. The MoT predictor has width 192, four blocks, four attention heads, and dropout 0.1. Its token sequence contains two history latents, one learned MSE state query, five clean action tokens, and ten action flow slots. Learned type and positional embeddings distinguish history, state query, clean action, and action flow tokens. The state readout is LayerNorm followed by a linear map to the 192-dimensional latent, while the action readout maps each action flow slot to a velocity vector in raw-action space.

For goal-conditioned experiments, the action readout concatenates each final action slot feature with the 192-dimensional goal latent, then applies three AdaLN-Zero MLP blocks with hidden width 512 and a 128-dimensional sinusoidal horizon condition. Goal dropout, when enabled, substitutes a learned null goal. The goal-free model has approximately 16.9M trainable parameters; the goal and horizon conditioned readout adds approximately 1.05M parameters, yielding 17.9M. Action sampling uses eight Euler evaluations. The reported MSE dynamics configuration uses an online encoder target, no state noise, no state flow integration, no momentum target encoder, and SIGReg with 1,024 projections and 17 knots. Its coefficient is λ_{sig} in Equation 5.

Tokens and Conditioning. The full MoT sequence is

$$\left[\mathbf{z}_{t-\Delta}, \mathbf{z}_t, \mathbf{q}_{t+K}^z, \mathbf{A}_t, \mathbf{a}^{(s)} \right]. \quad (9)$$

Here $\mathbf{A}_t$ and $\mathbf{a}^{(s)}$ expand into K clean and $2K$ noisy action tokens, respectively. The first three tokens use the state transformations, and both action groups use the action transformations. Each MoT layer has separate state and action query, key, value, output, and feedforward parameters, with one masked self-attention operation over the complete sequence. The learned state query receives neither state noise nor a flow time embedding. Flow time s modulates only the action flow slots through AdaLN-Zero.

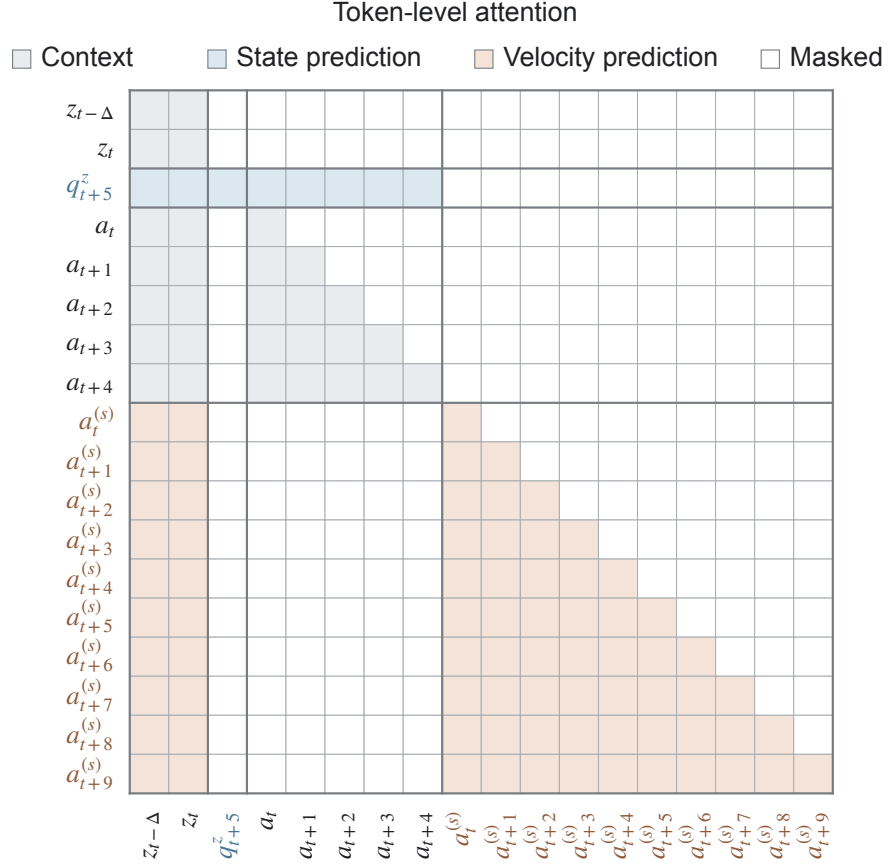


Figure 10: **Token-level MoT attention.** Expansion of Figure 3 for two history latents, one learned state query, five clean actions, and ten interpolated action tokens. Rows are queries; columns are keys. Filled cells permit attention; white cells are masked. Blue and brown mark latent state and action velocity prediction queries; gray marks context. Heavier lines separate token groups. Δ is the history stride, and action indices remain at the raw environment rate.

Attention Mask. Let $\mathcal{V}(x)$ denote the tokens visible to query x . Action prefixes begin at the current chunk’s start t ; padded history keys are excluded:

$$\begin{aligned}
 \mathcal{V}(z_i) &= \{z_{t-\Delta}, z_t\}, & i &\in \{t-\Delta, t\}, \\
 \mathcal{V}(q_{i+K}^z) &= \{z_{t-\Delta}, z_t, q_{i+K}^z, \mathbf{A}_t\}, \\
 \mathcal{V}(a_j) &= \{z_{t-\Delta}, z_t, \mathbf{a}_{\leq j}\}, & j &\in \{t, \dots, t+K-1\}, \\
 \mathcal{V}(a_j^{(s)}) &= \{z_{t-\Delta}, z_t, \mathbf{a}_{\leq j}^{(s)}\}, & j &\in \{t, \dots, t+2K-1\}.
 \end{aligned} \tag{10}$$

Clean action tokens therefore cannot relay target information to history or action flow tokens across layers.

B.1 BASELINE IMPLEMENTATIONS

The baselines in Section 4.1 use the released Stable World Model codebase (Maes et al., 2026a), retaining its environment wrappers, dataset interface, action bounds, and evaluation loop. Unless stated below, each method follows its official implementation and hyperparameters.¹ These configurations describe goal-conditioned comparisons, including start-to-end goal reaching in Figure 12.

¹Official repositories: DINO-WM, Stable World Model, and GC-IDM.

Methods share observations, actions, start and goal pairs, and success predicates. We list deviations from released configurations.

- **LeWM (IDM).** This is the GC-IDM (Nguyen et al., 2026) configuration: a pretrained LeWM is frozen, then a goal-conditioned inverse-dynamics model is post-trained. We follow its official two-stage implementation. Goal reaching samples a remaining horizon uniformly from 1 to 50 and clips it to the episode boundary. Start-to-end goal reaching uses the final image of the same trajectory as its only goal input and omits normalized-horizon conditioning. The two protocol-specific heads share one frozen latent cache. Compared with the official code, we move random goal sampling from minibatch loading to an equivalent precomputation stage, preserving the sampling distribution while reducing data-loading overhead.
- **LeWM (CEM).** We follow the official LeWM training and CEM evaluation settings (Maes et al., 2026b). On the original four goal-reaching benchmarks, our reproduced checkpoints perform similarly to the released checkpoints; all reported results use our checkpoints. Short-horizon and start-to-end goal reaching differ only in goal construction and evaluation horizon, and each protocol is evaluated over ten seeds.
- **DINO-WM.** We evaluate the released patch-token model and a matched CLS-token variant; tokenization is their only architectural difference. The four added tasks use ten training epochs, while the original four retain their four-epoch checkpoints. One checkpoint supports both goal-conditioned protocols. Start-to-end evaluation executes 25 environment steps between CEM replans.
- **GCBC.** We use the goal-conditioned behavior-cloning baseline from the released code-base (Maes et al., 2026a), with its pretrained DINOv2 encoder frozen and the policy trained for 20 rather than 100 epochs. Short-horizon goal reaching samples a uniformly later frame from the same episode. The start-to-end policy is trained separately and uses the final image of the same trajectory as its only goal input. At evaluation, each policy executes its predicted five-step action chunks without planning.

B.2 SYSTEM-LEVEL SIMPLICITY AND COST

Table 5 complements Section 2 with a comparison of training requirements and prediction capabilities.

Table 5: **World models and world action models.** FDM denotes forward dynamics conditioned on clean actions; Policy denotes learned action prediction. Entries describe reported base recipes.

Method	FDM	Policy	Joint FDM and Policy	End-to-end from scratch
<i>World models</i>				
DINO-WM (Zhou et al., 2024)	✓	✗	✗	✗
V-JEPA 2-AC (Assran et al., 2025)	✓	✗	✗	✗
LeWM (Maes et al., 2026b)	✓	✗	✗	✓
<i>World action models</i>				
UVA (Li et al., 2025)	✓	✓	✗	✗
UWM (Zhu et al., 2025)	✓	✓	✗	✗
DreamZero (Ye et al., 2026)	✗	✓	✗	✗
Fast-WAM (Yuan et al., 2026)	✗	✓	✗	✗
Cosmos 3 (NVIDIA, 2026)	✓	✓	✗	✗
LeWAM (ours)	✓	✓	✓	✓

Joint requires clean-action FDM and policy losses on the same sample in one training pass. From scratch includes visual encoding and prediction without pretrained or frozen components. UWM uses a frozen pretrained VAE even when its robot backbone is trained from scratch.

C DATASETS AND BENCHMARKS

Table 6 describes the eight environments evaluated in Section 4.1, their camera configurations, and the capability motivating each choice.

Table 6: Environment descriptions and selection motivations. Tasks, selected camera views, and evaluation motivations for the eight environments in Figure 5.

Environment	Description	Motivation
Two-Room	2D point navigation between rooms through a doorway. One scene view.	Test obstacle-aware latent planning in a simple setting with a narrow passage.
Reacher	Two-joint planar arm reaching visual goal configurations. One scene view.	Test representations of joint configuration and action-dependent motion, with separate collection regimes for data-quality comparisons.
Push-T	Planar pushing of a T-shaped block to a target position and orientation. One scene view.	Test contact-dependent dynamics and control learned from non-expert interactions.
Cube	3D robotic pick-and-place of a cube. One scene view.	Extend control beyond planar motion to grasping, lifting, and object placement.
DrawerCleanup	Dexterous drawer manipulation and object placement. Three views: scene and both wrists.	Test sequential manipulation of articulated objects. Wrist views expose local interactions hidden from the scene camera.
Transport	Two-arm payload transfer with a handover. Three views: scene and both wrists.	Test sustained bimanual coordination. Complementary views cover hand-object interactions occluded in the scene view.
ToolHang	Precise 3D assembly and tool hanging. One oblique scene view, approximately 45° .	Test whether partially observed image histories support useful latent representations for precise planning and control.
PointMaze-Large	2D point navigation in a large maze. One scene view.	Test obstacle-aware route composition and sustained planning over longer trajectories.

Data sources and collection regimes. DrawerCleanup and Transport use generated demonstrations from DexMimicGen (Jiang et al., 2025). ToolHang uses human demonstrations from robomimic (Mandlekar et al., 2021). Cube uses scripted expert demonstrations, and PointMaze-Large uses scripted navigation trajectories, both from OGBench (Park et al., 2025). Push-T uses non-expert interactions. Reacher has separate random-policy, goal-directed, and mixed collections. The mixed variant combines uniform random and goal-directed collections in equal proportions; the variants retain separate identities and are not pooled into a single result. The random-policy dataset has a reported mean of 201 frames per trajectory.

Table 7 collects the data regimes, recorded trajectory lengths, and scale of the complete source datasets, before experiment-specific splits. Its Reacher row reports the goal-directed collection, with a rounded mean of 143 frames; counts for the random and mixed variants are not included in this total. Recorded lengths describe the data, while Appendix D.1 specifies evaluation horizons and execution budgets. Recorded duration uses the native sampling rate before temporal subsampling; synchronized views of the same trajectories are counted once.

Table 7: **Simulation dataset properties and scale.** Collection regimes, trajectory counts, rounded mean and total recorded frames, native sampling rates, and nominal durations of the complete source datasets. The *Reacher* row covers the goal-directed collection.

Dataset	Collection regime	Traj.	Mean frames	Total frames	Hz	Hours
Two-Room	Noisy heuristic policy	10,000	92	920,809	10	25.58
Reacher (goal-directed)	Noisy goal-directed policy	17,504	143	2,498,208	25	27.76
Push-T	Non-expert interactions	18,685	125	2,336,736	10	64.91
Cube	Scripted expert demos	10,000	201	2,010,000	20	27.92
DrawerCleanup	Generated demos	1,026	291	298,235	20	4.14
Transport	Generated demos	1,029	408	419,712	20	5.83
ToolHang	Human demos	200	480	95,962	20	1.33
PointMaze-Large	Scripted navigation	1,000	1,000	1,000,000	10	27.78
Total		59,444		9,579,662		185.24

D EVALUATION PROTOCOL

For the comparisons in Section 4.1, unless stated otherwise, we report mean success and standard deviation over ten seeds with 50 episodes per seed. Methods within a protocol share observations, action limits, execution budgets, and success predicates. Latency excludes simulation and rendering.

D.1 GOALS AND SUCCESS CRITERIA

Short-horizon goal reaching. In Section 4.2, the initial state is sampled from an offline trajectory and the frame 25 steps later supplies the visual goal. Each controller receives 50 environment actions across all eight environments. We report first-hit success for comparison with the released LeWM evaluation (Maes et al., 2026b); the *Cube* predicate correction is detailed below.

TODO: Recap the results in Section 4.2 and give concrete numbers (e.g. , average improvement of x%).

Start-to-end goal reaching. Figure 12 uses a demonstration’s first observation as the start and its final observation as the goal. This terminal-image conditioning is distinct from the goal-free task-completion protocol in Appendix E.2.

Cube success predicate. The original *Cube* criterion checks whether the cube center lies within 0.04 m of the target, regardless of the gripper pose. For 25-step goals, this credits zero action on 42.7% of episodes (Table 8). The corrected criterion additionally requires the end effector to lie within 0.04 m of its goal-frame position, reducing this floor to 2.7%. Figure 11 illustrates the object-only failure case.



Figure 11: **A degenerate Cube success under short-horizon goal reaching.** Each row shows a policy rollout (left, frames over time) beside its goal frame (far right, after the divider). Throughout the execution the gripper never reaches the goal end-effector pose, yet the cube stays within the 0.04 m object-only threshold of the goal, so OGBench’s object-position-only predicate scores the episode a success. Our end-effector term (Table 8) rejects these degenerate successes by additionally requiring the gripper to reach the goal pose.

Environment	Raw action dim.	Short-horizon goal reaching		Start-to-end goal reaching	
		Zero-action	Random	Zero-action	Random
Two-Room	2	7.3 ± 0.9	25.3 ± 5.0	0.0 ± 0.0	2.7 ± 2.5
Reacher	2	3.3 ± 1.9	12.0 ± 1.6	0.0 ± 0.0	12.0 ± 4.9
Push-T	2	0.0 ± 0.0	1.3 ± 0.9	0.0 ± 0.0	0.0 ± 0.0
Cube	5	42.7 ± 6.6	44.0 ± 1.6		
Cube (+EEF)	5	2.7 ± 0.9	4.7 ± 0.9	4.7 ± 2.5	6.0 ± 1.6
DrawerCleanup	24	0.0 ± 0.0	6.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0
Transport	14	3.3 ± 0.9	2.7 ± 0.9	0.0 ± 0.0	0.0 ± 0.0
ToolHang	7	22.7 ± 2.5	26.0 ± 1.6	0.0 ± 0.0	0.0 ± 0.0
PointMaze	2	6.0 ± 0.0	9.3 ± 1.9	0.7 ± 0.9	6.0 ± 3.3

Darker shading indicates a higher control success rate.

Table 8: **Action dimensions and policy-floor controls.** Rows follow Figure 5, reading its first row and then its second row; Cube (+EEF) isolates the corrected end-effector predicate. Raw action dimensionality is reported beside zero-action and random-policy success rates (%) for short-horizon goal reaching (50 steps) and start-to-end goal reaching, averaged over three seeds. Because the EEF term changes only short-horizon evaluation, start-to-end controls are reported once in the Cube (+EEF) row. DrawerCleanup exposes 24 raw coordinates; the policy predicts 22 active dimensions after removing two constant joints. PointMaze uses the PointMaze-Large layout.

D.2 LEWAM PLANNING

The planner in Section 3.2 uses the frozen encoder and predictor with normalized actions. For a requested goal offset L_g on the dynamics grid, clipped to $[K, L]$, its rollout cost is

$$\mathcal{C}(\mathbf{a}) = \text{MSE}(\hat{\mathbf{z}}_{t+L_g}(\mathbf{a}), \mathbf{z}_g). \quad (11)$$

When $L_g = L$, this is Equation 7. The algorithm uses N candidates and J optimizer iterations; MPC executes the first action block before replanning from the next observation.

Gradient Planner Settings. The optimization objective includes a quadratic proximity penalty with weight λ_{TR} in normalized action coordinates:

$$\mathcal{J}_{\text{TR}}(\mathbf{a}; \mathbf{a}^{\text{init}}) = \mathcal{C}(\mathbf{a}) + \lambda_{\text{TR}} \|\mathbf{a} - \mathbf{a}^{\text{init}}\|_F^2. \quad (12)$$

The penalty discourages large departures from the initial plan but does not constrain policy support. Gradient updates use learning rate η and clip actions to the bound $a_{\max}$.

Algorithm 1 Global-to-local planning with LeWAM

Require: $\mathbf{z}_{\leq t}, \mathbf{c}_t, \mathbf{z}_g, L, L_g, N, J, \eta, \lambda_{\text{TR}}, a_{\max}$

Global search in the action distribution (one iteration)

```

1: for  $n = 1, \dots, N$  do
2:    $\mathbf{a}^{(n)} \leftarrow$  policy rollout from  $(\mathbf{z}_{\leq t}, \mathbf{c}_t)$  for  $L/K$  blocks
3:    $c_n \leftarrow \mathcal{C}(\mathbf{a}^{(n)})$  by rolling the candidate through  $f_z$ 
4: end for
5:  $\mathbf{a}^{\text{init}} \leftarrow$  candidate with the smallest  $c_n$   $\triangleright$  best-of- $N$  initialization
Gradient-based search
6:  $\mathbf{a} \leftarrow \mathbf{a}^{\text{init}}, \mathbf{a}^* \leftarrow \mathbf{a}^{\text{init}}$ 
7:  $c^* \leftarrow \mathcal{C}(\mathbf{a}^{\text{init}})$ 
8: for  $j = 0, \dots, J - 1$  do
9:    $\mathbf{a} \leftarrow \text{AdamStep}(\mathbf{a}, \nabla_{\mathbf{a}} \mathcal{J}_{\text{TR}}(\mathbf{a}; \mathbf{a}^{\text{init}}), \eta)$ 
10:   $\mathbf{a} \leftarrow \text{clip}(\mathbf{a}, -a_{\max}, a_{\max})$ 
11:   $c \leftarrow \mathcal{C}(\mathbf{a})$ 
12:  if  $c < c^*$  then
13:     $\mathbf{a}^* \leftarrow \mathbf{a}, c^* \leftarrow c$ 
14:  end if
15: end for
16: return  $\mathbf{a}^*$ 

```

Only the normalized action plan receives gradients; all model parameters remain frozen. The implementation uses $N = 32$ proposals, $J = 50$ Adam updates with learning rate 0.05, gradient-norm clipping at 10, and $a_{\max} = 3$ unless stated otherwise. Retaining the initial plan prevents degradation under the model cost, but does not guarantee improvement in the environment.

TODO: For Section 4.2, state that this is gradient planning with 32 samples, and provide a more detailed table with reactive and best-of-k scores here.

D.3 GOAL-HORIZON EVALUATION AND TIMING

Goal-horizon sweep. Table 9 gives the measurements in Figure 7 (Section 4.2). On Push-T, LeWAM uses SIGReg and the gradient-TR configuration; LeWM (CEM) (Maes et al., 2026b) uses the author-released checkpoint with the reported 300×30 CEM configuration. The goal is the recorded frame 25, 50, 75, or 100 steps after the initial frame. Each episode starts at frame 0, receives twice the goal horizon in environment actions, and replans every 25 steps. These are the reported solver configurations; matched-compute comparisons remain pending. Cube’s 25-step values reuse the sampled-start results in Figure 6. Longer horizons evaluate the original trained model zero-shot, without horizon-specific retraining. The preliminary LeWAM success estimate at 100 steps is approximately 72.7%; its standard deviation remains pending. The other longer-horizon points are numeric placeholders for visual review.

Table 9: **Goal-reaching success across horizons.** Success rate (%), mean $\pm$ standard deviation. Measured values for Figure 7. Cube’s 25-step values reuse the sampled-start records in Figure 6; the $\approx 72.7\%$ entry is a preliminary zero-shot estimate without a reported standard deviation. Other missing entries remain pending.

Task	Method	Goal horizon (steps)			
		25	50	75	100
Push-T	LeWAM	98.0 ± 2.0	71.3 ± 3.1	46.7 ± 6.4	45.3 ± 6.4
	LeWM (CEM)	68.7 ± 3.1	12.0 ± 4.0	3.3 ± 2.3	2.7 ± 1.2
Cube	LeWAM	100.0 ± 0.0	TBD	TBD	≈ 72.7
	LeWM (CEM)	56.7 ± 6.1	TBD	TBD	TBD

Timing scope. Figure 1(c), discussed in Section 4.5, reports planning throughput as the reciprocal of per-action planning latency. Hardware, timing boundaries, and solver configurations remain to be finalized.

E ADDITIONAL RESULTS

E.1 START-TO-END GOAL REACHING

Figure 12 complements the main goal-reaching results (Section 4.2) using the start-to-end protocol in Appendix D.1.

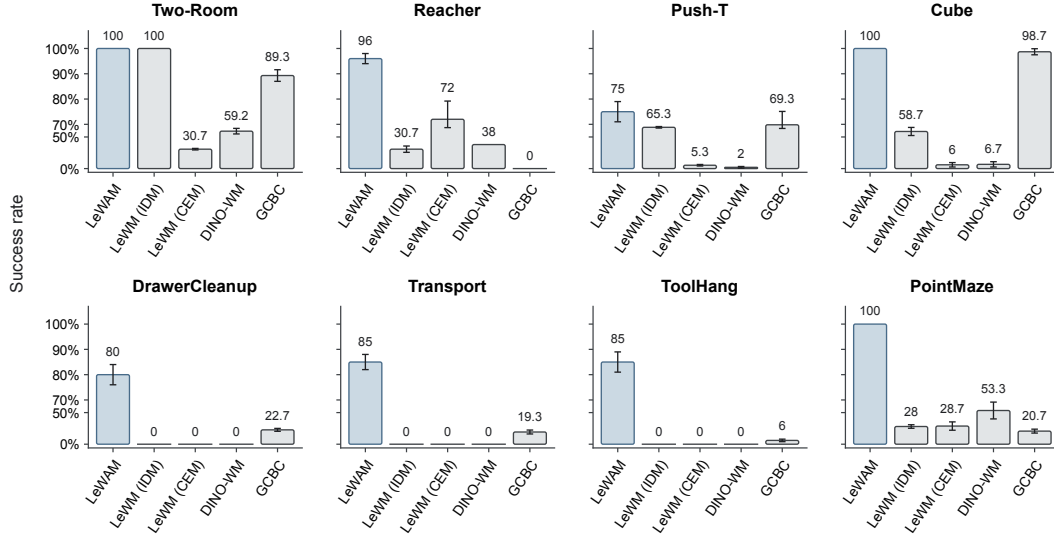


Figure 12: **Start-to-end goal reaching.** All methods receive the trajectory’s terminal image as the goal. Error bars denote one standard deviation. All panels use a piecewise linear vertical axis, with fourfold spacing per percentage point above 70%. Only the LeWAM results on DrawerCleanup and Transport use earlier measurements pending final results.

E.2 BEHAVIOR CLONING

The supplementary task-completion comparison in Section 4.3 uses observation history without a goal image or goal-horizon input. LeWAM directly executes its action outputs with c_t omitted and retains the joint training objective. Table 1 compares it with Diffusion Policy (DP-T) (Chi et al., 2023). The available ToolHang and DrawerCleanup results use single-view LeWAM; Cube and Transport results remain pending.

F PHYSICAL-ROBOT SETUP AND EVALUATION

This section documents the physical-robot experiments in Section 4.6.

F.1 PLATFORM AND DATA COLLECTION

Demonstrations are collected by joint-space teleoperation from a reBot Arm 102 leader to a reBot B601-DM follower through LeRobot (Cadene et al., 2026). The follower uses impedance control with gravity feedforward. One proficient operator collects successful demonstrations without intentional exploration. Between episodes, both arms return to the calibration zero pose before recording resumes.

A ZED-Mini records the scene at 672×376 pixels per eye, and a wrist-mounted Logitech C270 records at 640×360 pixels. The model uses the left scene view and wrist view; the right stereo view is recorded but unused. At 30 Hz, a Raspberry Pi 5 stores the three video streams, seven-dimensional joint targets as actions, and follower position, torque, and velocity as a 21-dimensional state.

Table 10: Datasets for the physical-robot tasks in Section 4.6. The third task and its collection statistics remain TBD.

Task	Episodes	Frames	Duration	Mean episode
Pick and Place T	200	52,297	29 min	8.72 s
Push-T	100	61,996	34 min	20.67 s
TBD	TBD	TBD	TBD	TBD

F.2 MODEL INTERFACES

The LeWAM models in Section 4.6 use the state MSE and action flow matching objectives from Section 3.1. In the frozen V-JEPA 2.1 variant, attentive pooling and a BatchNorm MLP reduce each camera frame to one summary token. Proprioceptive state and previous actions are linearly projected from raw vectors to the model dimension. Learned queries initialize the future-camera and action streams.

F.3 CLOSED-LOOP EVALUATION

For Section 4.6, the robot sends its observation window to a remote inference machine, receives an action chunk, and executes it at 30 Hz under the receding-horizon schedule. A human evaluator labels each episode as success or failure. Each method is evaluated in 30 trials per task.